

Data Structures

Algorithms

STACK

PUSH (A, TOP, VAL)

1. Check for Overflow

if $TOP == N-1$ then

Print "Stack is Overflow"

exit

end if

2. Increment TOP

$TOP = TOP + 1$

3. Insert Element

$A[TOP] = VAL$

4. End

STACK

POP (A, TOP)

1. Check for Underflow

if $TOP == -1$ then

Print "Stack is Underflow"

exit

end if

2. Fetch Value at TOP Position

Print "Value at A[TOP] is deleted"

3. Delete Element

$TOP = TOP - 1$

4. End

STACK

PEEP (A, TOP)

1. Check for Underflow
 if TOP == -1 then
 Print "Stack is Underflow"
 exit
 end if
2. Take Value from TOP Position
 Value = A[TOP]
3. Return Value
 Return (Value)
4. End

INFIX to POSTFIX

INFIX_TO_POSTFIX (expr)

1. Append “)” to expr
2. Push “(“ into STACK
3. Repeat following steps until all characters of Infix “expr” are scanned...
 1. If Next Character is “(“ then PUSH it into STACK
 2. If next character is OPERAND then Append it to POSTFIX expression
 3. If next character is “)” then
 1. POP all character from STACK until “(“ is encountered and Append them to POSTFIX expression
 2. Discard “(“ from STACK, don't add to POSTFIX expression

INFIX to POSTFIX

4. If next character is OPERATOR – O then
 4. POP each operator from STACK until next operator has higher or same precedence than OPERATOR – O.
 5. PUSH OPERATOR – O into STACK.
5. Repeatedly POP the STACK and Append to POSTFIX expression till STACK becomes Underflow
6. End

COMPUTE REVERSE POLISH NOTATION

COMPUTE_REV_POLISH_NOTATION(expr)

1. Append “)” to expr
2. Push “(“ into STACK and Repeat STEP-3 and STEP-4 until “)” is encountered from “expr”
3. If and OPERAND is encountered then PUSH it into STACK
4. If OPERATOR – O is encountered then
 1. POP Two elements from STACK into variable A and B
 2. Perform $B \ O \ A$ or $A \ O \ B$ operation
 3. PUSH Result into stack
5. Set Final Result on TOP most position of the STACK
6. End

COMPUTER REVERSE POLISH NOTATION

Example: $(9 - ((3 * 4) + 8) / 4) \Rightarrow 9 \ 3 \ 4 \ * \ 8 \ + \ 4 \ / \ -$

Step	Character	Stack	
2	(	(	
3	9	(9	
3	3	(9 3	
3	4	(9 3 4	
4	*	(9 12	(bcz, pop 3 and 4, do $4 * 3 = 12$, push into stack
3	8	(9 12 8	
4	+	(9 20	(bcz pop 12 and 8, do $8 + 12 = 20$, push into stack
3	4	(9 20 4	
4	/	(9 5	(bcz pop 20 and 4, do $20/4=5$, push into stack
4	-	(4	(bcz pop 9 and 5, do $9-5$, push into stack
5	-	-	Result is Topmost element of Stack. Result = 4

INFIX TO PREFIX CONVERSION

INFIX_TO_PREFIX(expr)

[OR CONVERT_TO_POLISH_NOTATION(expr)]

1. Reverse the Infix “expr” and store into “RevExpr”
2. Convert “RevExpr” to it’s Postfix expression and store into “POSTFIXexpr”
3. Reverse “POSTFIXexpr” and store into “PREFIXexpr”
4. End

RECURSIVE PROBLEMS

CALUCATE_FACTORIAL (NO1)

1. Check Termination Condition

 if NO1 == 1 then

 return (1);

 end if

2. Calculate Factorial Recursively

 return (NO1 * CALCULATE_FACTORIAL (NO1 – 1))

3. End

RECURSIVE PROBLEMS

CALUCATE_GCD (NO1, NO2)

1. Check Termination Condition
 - if $NO1 \% NO2 == 0$ then
 - return (NO2);
 - end if
2. Calculate GCD Recursively
 - Return (GCD (NO2, $NO1 \% NO2$))
3. End

RECURSIVE PROBLEMS

CALUCATE_EXPONENT (NO1, NO2)

1. Check Termination Condition

if NO2 == 1 then

return (NO1);

2. Calculate EXPONENT Recursively

else if (NO2 % 2 != 0) then

return (NO1 * CALCUALTE_EXPONENT(NO1, NO2-1))

else

return (POWER (CALCULATE_EXPONENT(NO1, NO2 / 2), 2)

end if

3. End

RECURSIVE PROBLEMS

CALUCATE_FIBONACCI (NO1)

1. Check Termination Condition

if NO1 == 0 then

return (0);

else if NO1 == 1 then

return (1)

end if

2. Calculate FIBONACCI Recursively

return (CALCULATE_FIBONACCI (NO1 -1) +

CALCULATE_FIBONACCI (NO1 – 2))

3. End

RECURSIVE PROBLEMS

TOWER_OF_HONOI (N, BEG, AUX, END)

1. If $N == 1$ then
 Write "BEG -> END"
 exit
2. If $N > 1$ then
 TOWER_OF_HONOI (n-1 , BEG, END, AUX)
 Write "BEG -> END"
 TOWER_OF_HONOI (n-1, AUX, BEG, END)
end if
3. END

```
towerOfHanoi(n, from_rod, to_rod, aux_rod)
{
    if (n == 1)
    {
        printf "Move" n "from rod" from_rod "to rod" to_rod
        return;
    }
    towerOfHanoi(n-1, from_rod, aux_rod, to_rod);
    printf "Move" n "from rod" from_rod "to rod" to_rod
    towerOfHanoi(n-1, aux_rod, to_rod, from_rod);
}
```

Initial Call:

```
towerOfHanoi(n, 'A', 'C', 'B');
```

QUEUE

INSERTION_INTO_QUEUE (Q, FRONT, REAR, VAL, MAX)

1. Check for Queue is full or not
 if $REAR == MAX - 1$ then
 Write "Queue is Full"
 exit
 end if
2. Read Data "VAL" from User
3. Insert value "VAL" in Queue – Q
 if $FRONT == -1$ then
 $FRONT = FRONT + 1$
 $REAR = REAR + 1$
 $Q[REAR] = VAL$
 else
 $REAR = REAR + 1$
 $Q[REAR] = VAL$
 end if
4. End

QUEUE

DELETION_FROM_QUEUE (Q, FRONT, REAR)

1. Check for Queue is empty or not
 if FRONT == -1 then
 Write "Queue is Empty"
 exit
 end if
2. Check if only one element in Queue – Q
 if FRONT == REAR then
 Q [FRONT] = NULL
 FRONT = REAR = -1
 end if
3. Delete Element from Queue – Q
 Q [FRONT] = null
 FRONT = FRONT + 1
4. End

CIRCULAR QUEUE

CIRCULAR_QUEUE_INSERT (Q, FRONT, REAR, VAL, MAX)

1. Check for Queue – Q is full or not
 if (FRONT==0 and REAR == MAX-1) OR (FRONT==REAR+1) then
 Write “Queue is FULL”
 end if
2. Take Data from User into variable “VAL”
3. Insert into Queue – Q
 if FRONT == -1 and REAR == -1 then
 FRONT = REAR = 0
 else if FRONT != 0 and REAR == MAX – 1 then
 REAR = 0
 else
 REAR = REAR + 1
 end if
 Q[REAR] = VAL
4. End

CIRCULAR QUEUE

CIRCULAR_QUEUE_DELETE (Q, FRONT, REAR, MAX)

1. Check for Queue – Q is empty or not

 if FRONT == -1 then

 Write "Queue is EMPTY"

 end if

2. Delete Element from Queue - Q

 if FRONT == REAR then

 FRONT = REAR = -1

 else if FRONT == MAX-1 then

 FRONT = 0

 else

 FRONT = FRONT + 1

 end if

4. End

DOUBLE ENDED QUEUE (DEQUE)

DEQUE_INSERT_RIGHT (Q, LEFT, RIGHT, MAX, VAL)

1. *Check for Queue – Q is FULL or not*
 - if (LEFT==0 and RIGHT==MAX-1) or (LEFT==RIGHT+1)
 - WRITE “Deque – Q is overflow”
 - exit
 - end if
2. *Input Data Value from User into “VAL”*
3. *Insert into Deque – Q*
 - if LEFT == -1 then
 - LEFT = RIGHT = 0
 - else if RIGHT == MAX – 1 then
 - RIGHT=0
 - else
 - RIGHT = RIGHT + 1
 - end if
4. *Insert Value at Position RIGHT*
 - Q [RIGHT] = VAL
4. *END*

DOUBLE ENDED QUEUE (DEQUE)

DEQUE_INSERT_LEFT (Q, LEFT, RIGHT, MAX, VAL)

1. Check for Queue – Q is FULL or not
 if (LEFT==0 and RIGHT==MAX-1) or (LEFT==RIGHT+1)
 WRITE “Deque – Q is overflow”
 exit
 end if
2. Input Data Value from User into “VAL”
3. Insert into Deque – Q
 if LEFT == -1 then
 LEFT = RIGHT = 0
 else if LEFT== 0 then
 LEFT=MAX-1
 else
 LEFT= LEFT - 1
 end if
4. Q [LEFT] = VAL
5. END

DOUBLE ENDED QUEUE (DEQUE)

DEQUE_DELETE_RIGHT (Q, LEFT, RIGHT, MAX)

1. Check for Queue – Q is EMPTY or not
 if (LEFT== -1 and RIGHT== -1)
 WRITE “Deque – Q is EMPTY”
 exit
 end if
2. Delete element from Deque – Q
 if LEFT == RIGHT then
 LEFT = RIGHT = -1
 else if RIGHT == 0 then
 RIGHT = MAX – 1
 else
 RIGHT = RIGHT - 1
 end if
4. END

DOUBLE ENDED QUEUE (DEQUE)

DEQUE_DELETE_LEFT (Q, LEFT, RIGHT, MAX)

1. Check for Queue – Q is EMPTY or not
 if (LEFT== -1 and RIGHT== -1)
 WRITE “Deque – Q is EMPTY”
 exit
 end if
2. Delete element from Deque – Q
 if LEFT == RIGHT then
 LEFT = RIGHT = -1
 else if LEFT = MAX - 1 then
 LEFT = 0
 else
 LEFT = LEFT + 1
 end if
4. END

PRIORITY QUEUE

PRIORITY_QUEUE_INSERT

(Q, MAXP, MAX, VAL, PRIORITY, FRONT[], REAR[])

1. Check for OVERFLOW

for (k = 1 to MAXP)

if (FRONT[k] == 0 and REAR[k] == MAX-1)

or (FRONT[k] == REAR[k]+1) then

else

Keep 'k' for insertion

end if

end for

2. Take data value from user into "VAL" and priority into "k"

PRIORITY QUEUE

3. Insert Value into Priority Queue - Q

If (FRONT[k] == -1) then

 FRONT[k] = REAR[k] = 0

else if (FRONT[k] != 0 and REAR[k] == MAX - 1) then

 REAR[k] = 0

else

 REAR[k] = REAR[k] + 1

end if

Q [k] [REAR [k]] = VAL

4. END

PRIORITY QUEUE

PRIORITY_QUEUE_DELETE

(Q, MAXP, MAX, FRONT[], REAR[])

1. Find NON-EMPTY Queue with Highest Priority
 for k=1 to MAXP
 if (FRONT[k] != -1) then
 break;
 end if
 end for
2. Delete value from Non-Empty Queue...
 if FRONT[k] == REAR [k] then
 FRONT[k] = REAR[k] = -1

PRIORITY QUEUE

else if $\text{FRONT}[k] == \text{MAX}-1$ then

$\text{FRONT}[k] = 0$

else

$\text{FRONT}[k] = \text{FRONT}[k] + 1$

end if

4. END

LINK LIST - SINGLY LINK LIST

INSERT_BEGINNING (START, VAL)

1. *Declare PTR*

struct node * PTR

2. *Set PTR = (struct node*) malloc(sizeof (struct node))*

3. *Check Memory is AVAILABLE or Not*

if PTR == NULL then

WRITE "Memory is not available"

exit

end if

4. *Set PTR->data = VAL*

PTR->next = START

5. *Set START = PTR*

6. *End*

LINK LIST - SINGLY LINK LIST

INSERT_AT_LAST (START, VAL)

1. *Declare PTR*

struct node * PTR

2. *Set PTR = (struct node*) malloc(sizeof (struct node))*

3. *Check Memory is AVAILABLE or Not*

if PTR == NULL then

WRITE "Memory is not available"

exit

end if

4. *Set PTR->data = VAL*

PTR->next = NULL

LINK LIST - SINGLY LINK LIST

5. *Find Last Position*

if START == NULL then

START = PTR

else

struct node *tmp = START

while (tmp -> next != NULL) then

tmp = tmp->next

end while

end if

6. *Set new Value*

tmp->next = PTR

7. *END*

LINK LIST - SINGLY LINK LIST

INSERT_AT_LOCATION (START, VAL, NUM)

1. *Declare PTR*

struct node * PTR

2. *Set PTR = (struct node*) malloc(sizeof (struct node))*

3. *Check Memory is AVAILABLE or Not*

if PTR == NULL then

WRITE "Memory is not available"

exit

end if

4. *Set PTR -> data = VAL*

PTR -> next = NULL

5. *Check List is Empty*

if START == NULL then set START = PTR

LINK LIST - SINGLY LINK LIST

6. *Find Proper Position*

set TEMP = START

set PRETEMP = START

while PRETEMP -> data != num

 PRETEMP = TEMP

 TEMP = TEMP -> next

end while

7. *Set new node "PTR" at found position*

PTR -> next = PRETEMP -> next

PRETEMP -> next = PTR

8. *END*

LINK LIST - SINGLY LINK LIST

DELETE_FIRST(START)

1. *Check for Empty*

if START == NULL then

Write "LIST IS EMPTY"

exit

end if

2. *Delete Node if list is not empty*

PTR = START

START = START -> next

Write "Node PTR -> data is Deleted"

3. *End*

LINK LIST - SINGLY LINK LIST

DELETE_LAST(START)

1. *Check for Empty*

```
    if START == NULL then
        Write "LIST IS EMPTY"
        exit
    end if
```

2. *Move to last Node*

```
    PTR = START
    while PTR -> next != NULL then
        PREVPTR = PTR
        PTR = PTR -> next
    end while
```

2. *Delete Last Node*

```
    PREVPTR -> next = NULL
    Write "Node PTR -> data is Deleted"
```

3. *End*

LINK LIST - SINGLY LINK LIST

DELETE_BY_VALUE(START , NUM) [DELETE_INBETWEEN(START, NUM)]

1. *Check for Empty*

```
    if START == NULL then
        Write "LIST IS EMPTY"
        exit
    end if
```

2. *Search Node having data NUM*

```
    PTR = START
    PREVPTR = START
    while PTR -> data != NUM then
        PREVPTR = PTR
        PTR = PTR -> next
    end while
```

3. *Delete Node*

```
    PREVPTR -> next = PTR -> next
    FREE (PTR)
```

4. *End*

LINK LIST - CIRCULAR SINGLY LINK LIST

INSERT_BEGINNING_CSLIST (START, VAL)

1. *Declare PTR*
 struct node * PTR
2. *Set PTR = (struct node*) malloc (sizeof (struct node))*
3. *Check Memory is AVAILABLE or Not*
 if PTR == NULL then
 WRITE "Memory is not available"
 exit
 end if
4. *Set PTR -> data = VAL*
 PTR -> next = NULL
5. *Check if it's First Node*
 if START == NULL then
 START = PTR;
 START -> next = START
 end if

LINK LIST - CIRCULAR SINGLY LINK LIST

6. *If that's not first node*

TEMP = START

while TEMP->next != START then

TEMP = TEMP -> next

end while

TEMP -> next = PTR

PTR -> next = START

START = PTR

7. *END*

LINK LIST - CIRCULAR SINGLY LINK LIST

INSERT_LAST_CSLIST (START, VAL)

1. *Declare PTR*
 struct node * PTR
2. *Set PTR = (struct node*) malloc (sizeof (struct node))*
3. *Check Memory is AVAILABLE or Not*
 if PTR == NULL then
 WRITE "Memory is not available"
 exit
 end if
4. *Set PTR -> data = VAL*
 PTR -> next = NULL
5. *Check if it's First Node*
 if START == NULL then
 START = PTR;
 START -> next = START
 end if

LINK LIST - CIRCULAR SINGLY LINK LIST

6. *If that's not first node*

TEMP = START

while TEMP->next != START then

TEMP = TEMP -> next

end while

TEMP -> next = PTR

PTR -> next = START

7. *END*

LINK LIST - CIRCULAR SINGLY LINK LIST

DELETE_BEGINNING_CSLIST(START)

1. *Check for Empty*

if START == NULL then

Write "LIST IS EMPTY"

exit

end if

2. *Move to the Last Node*

TEMP = START

while TEMP -> next != NULL then

TEMP = TEMP -> next

end while

3. *Delete Node*

TEMP -> next = START -> next

START = TEMP -> next

4. *End*

LINK LIST - CIRCULAR SINGLY LINK LIST

DELETE_END_CSLIST(START)

1. *Check for Empty*

```
    if START == NULL then
        Write "LIST IS EMPTY"
        exit
    end if
```

2. *Move to last Node*

```
    TEMP = START
    while TEMP -> next != START then
        PREVTEMP = TEMP
        TEMP = TEMP -> next
    end while
```

2. *Delete Last Node*

```
    PREVPTR -> next = START
    free (TEMP)
```

3. *End*

LINK LIST - DOUBLY LINK LIST

INSERT_BEGINNING_DLLIST (START, VAL)

1. *Declare PTR*

struct node * PTR

2. *Set PTR = (struct node*) malloc (sizeof (struct node))*

3. *Check Memory is AVAILABLE or Not*

if PTR == NULL then

WRITE "Memory is not available"

exit

end if

4. *Set PTR -> data = VAL*

PTR -> next = NULL

PTR -> prev = NULL

LINK LIST - DOUBLY LINK LIST

5. *Check if it's First Node*

if $START == NULL$ then

$START = PTR;$

end if

6. *If that's not first node*

$START \rightarrow prev = PTR$

$PTR \rightarrow next = START$

$START = PTR$

7. *END*

LINK LIST - DOUBLY LINK LIST

INSERT_END_DLLIST (START, VAL)

1. *Declare PTR*

struct node * PTR

2. *Set PTR = (struct node*) malloc (sizeof (struct node))*

3. *Check Memory is AVAILABLE or Not*

if PTR == NULL then

WRITE "Memory is not available"

exit

end if

4. *Set PTR -> data = VAL*

PTR -> next = NULL

PTR -> prev = NULL

LINK LIST - DOUBLY LINK LIST

5. *Check if it's First Node*
 if START == NULL then
 START = PTR;
 end if
6. *Find Proper Position at Last*
 TEMP = START
 while (TEMP->next != NULL)
 TEMP = TEMP -> next
 end while
7. *Insert Element*
 PTR -> prev = TEMP
 TEMP -> next = PTR
 PTR -> next = NULL
8. *END*

LINK LIST - DOUBLY LINK LIST

INSERT_INBETWEEN_DLLIST (START, VAL, NUM)

1. *Declare PTR*

struct node * PTR

2. *Set PTR = (struct node*) malloc (sizeof (struct node))*

3. *Check Memory is AVAILABLE or Not*

if PTR == NULL then

WRITE "Memory is not available"

exit

end if

4. *Set PTR -> data = VAL*

PTR -> next = NULL

PTR -> prev = NULL

LINK LIST - DOUBLY LINK LIST

5. *Check if it's First Node*
 if START == NULL then
 START = PTR;
 end if
6. *Find Proper Position at Last*
 TEMP = START
 while (TEMP -> data != NUM)
 TEMP = TEMP -> next
 end while
7. *Insert Element*
 PTR -> next = TEMP -> next
 PTR -> prev = TEMP
 TEMP -> next -> prev = PTR
 TEMP -> next = PTR
8. *END*

LINK LIST – DOUBLY LINK LIST

DELETE_BEGINNING_DLL(START)

1. *Check for Empty*

if START == NULL then

Write "LIST IS EMPTY"

exit

end if

2. *Delete first Node*

PTR = START

START = START -> next

START -> prev = NULL

Write "Node PTR -> data is Deleted"

3. *End*

LINK LIST – DOUBLY LINK LIST

DELETE_END_DLL(START)

1. *Check for Empty*

```
    if START == NULL then
        Write "LIST IS EMPTY"
        exit
    end if
```

2. *Move to LAST Node*

```
    TEMP = START
    while TEMP->next != NULL
        TEMP = TEMP -> next
    end while
```

3. *Delete Last Node*

```
    TEMP -> prev -> next = NULL
    FREE (TEMP)
```

4. *END*

LINK LIST – DOUBLY LINK LIST

DELETE_INBETWEEN_DLL(START, NUM)

1. *Check for Empty*

```
    if START == NULL then
        Write "LIST IS EMPTY"
        exit
    end if
```

2. *Move to SPECIFIC Node*

```
    TEMP = START
    while TEMP -> data != NUM
        TEMP = TEMP -> next
    end while
```

3. *Delete Node*

```
    TEMP -> prev -> next = TEMP -> next
    TEMP -> next -> prev = TEMP -> prev
    FREE (TEMP)
```

4. *END*

CAN YOU MAKE
10
FROM NUMBERS
1, 1, 5, 8
AND OPERATORS
+ - x ÷ ()

(use numbers once only)

Solution : $8 / (1 - 1/5) = 10$

LINK LIST - CIRCULAR DOUBLY LINK LIST

INSERT_BEGINNING_CDLLIST (START, VAL)

1. *Declare PTR*

struct node * PTR

2. *Set PTR = (struct node*) malloc (sizeof (struct node))*

3. *Check Memory is AVAILABLE or Not*

if PTR == NULL then

WRITE "Memory is not available"

exit

end if

4. *Set PTR -> data = VAL*

PTR -> next = NULL

PTR -> prev = NULL

LINK LIST - CIRCULAR DOUBLY LINK LIST

5. *Check if it's First Node*

if $START == NULL$ then

$START = PTR;$

end if

6. *If that's not first node, Set temp to Last Node*

$temp = START \rightarrow prev$

7. *Insert Node*

$START \rightarrow prev = PTR$

$temp \rightarrow next = PTR$

$PTR \rightarrow next = START$

$PTR \rightarrow prev = temp$

$START = PTR$

8. *END*

LINK LIST - CIRCULAR DOUBLY LINK LIST

INSERT_END_CDLLIST (START, VAL)

1. *Declare PTR*

struct node * PTR

2. *Set PTR = (struct node*) malloc (sizeof (struct node))*

3. *Check Memory is AVAILABLE or Not*

if PTR == NULL then

WRITE "Memory is not available"

exit

end if

4. *Set PTR -> data = VAL*

PTR -> next = NULL

PTR -> prev = NULL

LINK LIST - CIRCULAR DOUBLY LINK LIST

5. *Check if it's First Node*

if $START == NULL$ then

$START = PTR;$

end if

6. *If that's not first node, Set temp to Last Node*

$temp = START \rightarrow prev$

7. *Insert Node*

$START \rightarrow prev = PTR$

$temp \rightarrow next = PTR$

$PTR \rightarrow next = START$

$PTR \rightarrow prev = temp$

8. *END*

LINK LIST - CIRCULAR DOUBLY LINK LIST

INSERT_INBETWEEN_CDLLIST (START, VAL, NUM)

1. *Declare PTR*

struct node * PTR

2. *Set PTR = (struct node*) malloc (sizeof (struct node))*

3. *Check Memory is AVAILABLE or Not*

if PTR == NULL then

WRITE "Memory is not available"

exit

end if

4. *Set PTR -> data = VAL*

PTR -> next = NULL

PTR -> prev = NULL

LINK LIST - CIRCULAR DOUBLY LINK LIST

5. *Check if it's First Node*

if START == NULL then

START = PTR;

end if

6. *Find Proper Position at Last*

TEMP = START

while (TEMP -> data != NUM)

TEMP = TEMP -> next

end while

7. *Insert Element*

PTR -> next = TEMP -> next

PTR -> prev = TEMP

TEMP -> next -> prev = PTR

TEMP -> next = PTR

8. *END*

LINK LIST – CIRCULAR DOUBLY LINK LIST

DELETE_BEGINNING_CDLLIST(START)

1. *Check for Empty*

if START == NULL then

Write “LIST IS EMPTY”

exit

end if

2. *Delete first Node*

START -> next -> prev = START -> prev

START -> prev -> next = START -> next

START = START -> next

3. *End*

LINK LIST – CIRCULAR DOUBLY LINK LIST

DELETE_END_CDLLIST(START)

1. *Check for Empty*

if $START == NULL$ then

Write "LIST IS EMPTY"

exit

end if

2. *Delete Last Node*

$TEMP = START \rightarrow prev$

$PRETEMP = TEMP \rightarrow prev$

$PRETEMP \rightarrow next = START$

$START \rightarrow prev = PRETEMP$

$FREE(TEMP)$

3. *END*

LINK LIST – CIRCULAR DOUBLY LINK LIST

DELETE_INBETWEEN_DLL(START, NUM)

1. *Check for Empty*

if START == NULL then
 Write "LIST IS EMPTY"
 exit
end if

2. *Move to SPECIFIC Node*

TEMP = START
while TEMP -> data != NUM
 TEMP = TEMP -> next
end while

3. *Delete Node*

TEMP -> prev -> next = TEMP -> next
TEMP -> next -> prev = TEMP -> prev
FREE (TEMP)

4. *END*

Place the numbers 1 to 9 to make a valid equation

			-		66
+		×		-	=
13		12		11	10
×		+		+	-
:		+		×	:

$$9 + (13 \times 4 \div 8) + 5 + (12 \times 6) - 7 - 11 + (1 \times 3 \div 2) - 10 = 66$$

APPLICATION OF STACK & QUEUE

Application of STACK

1. Reversing a list
2. Parentheses checker
3. Conversion of an infix expression into a postfix expression
4. Evaluation of a postfix expression
5. Conversion of an infix expression into a prefix expression
6. Evaluation of a prefix expression
7. Recursion
8. Tower of Hanoi

APPLICATION OF STACK & QUEUE

Application of QUEUE

1. Queues are widely used as waiting lists for a single shared resource like printer, disk, CPU.
2. Queues are used to transfer data asynchronously between two processes (IO buffers), e.g., pipes, file IO, sockets.
3. Queues are used as buffers on MP3 players AND Playlist
4. Queues are used in operating system for handling interrupts. When programming a real-time system that can be interrupted, for example, by a mouse click, it is necessary to process the interrupts immediately, before proceeding with the current job. If the interrupts have to be handled in the order of arrival, then a FIFO queue is the appropriate data structure.

APPLICATION OF LINK LIST

1. Linked lists can be used to represent polynomials and the different operations that can be performed on them.

For Example: $6x^3 + 9x^2 + 7x + 1$

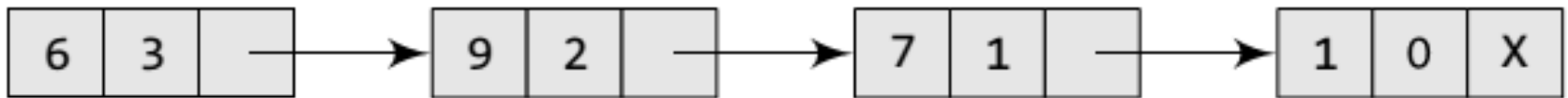


Figure 6.74 Linked representation of a polynomial

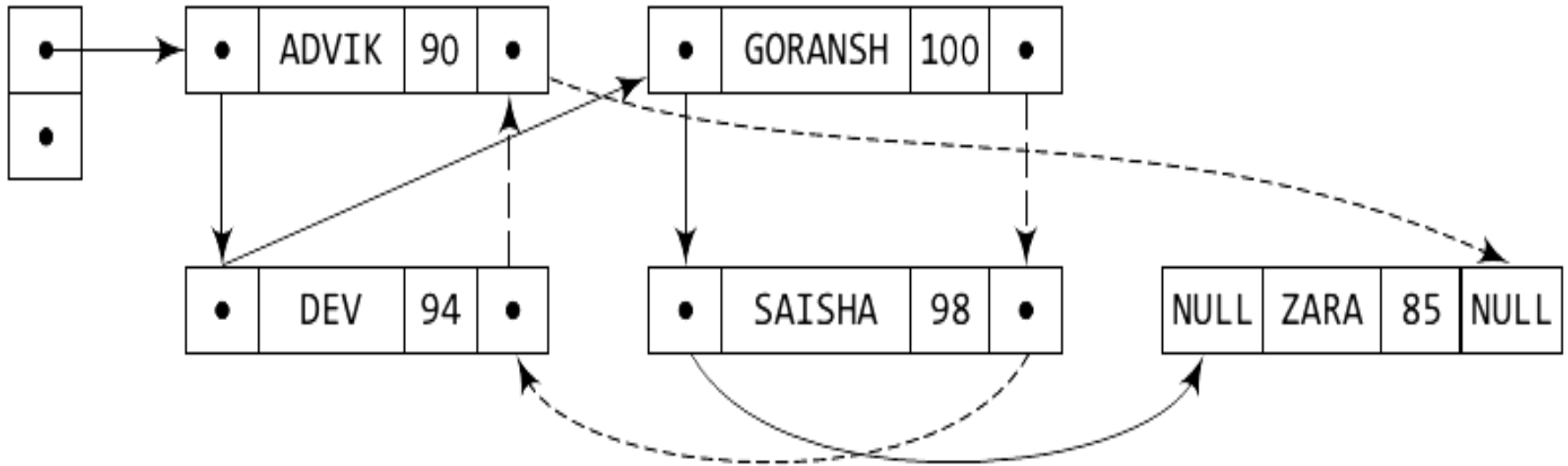
Exercise: Write A Program for the Polynomial Expression Addition using Link List

SPECIAL LINK LIST – 1. MULTI LINK LIST

- In a multi-linked list, each node can have “n” number of pointers to other nodes.
- A doubly linked list is a special case of multi-linked lists.
- Difference between Multi Link List and Doubly Link List
 - A doubly linked list has exactly two pointers. But a node in the multi-linked list can have any number of pointers.
 - In a doubly linked list, pointers are exact inverses of each other, i.e., for every pointer which points to a previous node there is a pointer which points to the next node. This is not true for a multi-linked list.
- Multi-linked lists are generally used to organize multiple orders of one set of elements.
- All the operations can be performed like Doubly Link List

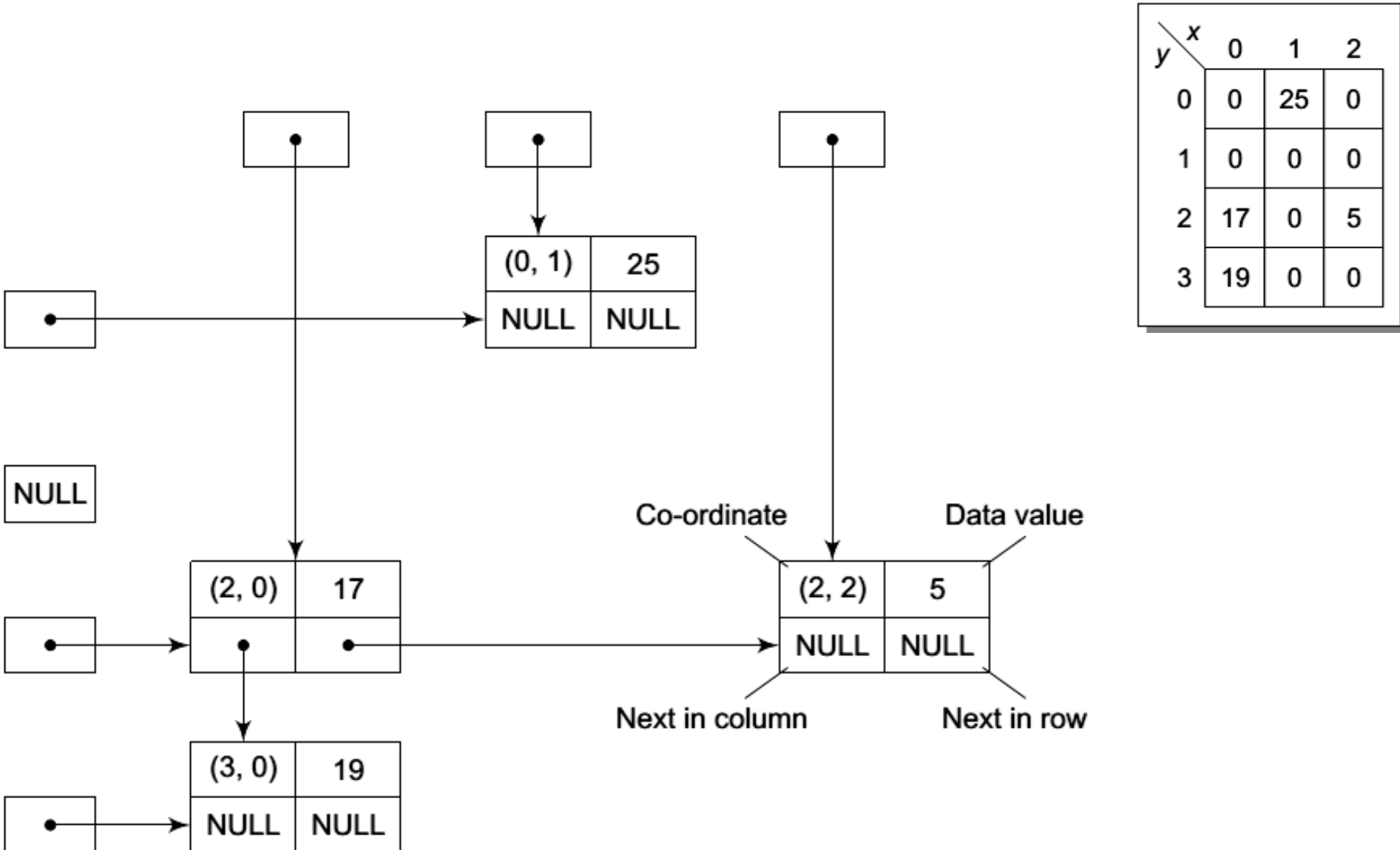
SPECIAL LINK LIST – 1. MULTI LINK LIST

- For Example, a Class of Student,
 - organize the nodes alphabetically (according to the name)
 - organize the nodes according to decreasing order of marks



SPECIAL LINK LIST – 1. MULTI LINK LIST

- For Example, Sparse Matrix Representation using Multi Link List



SPECIAL LINK LIST – 2. HEADER LINK LIST

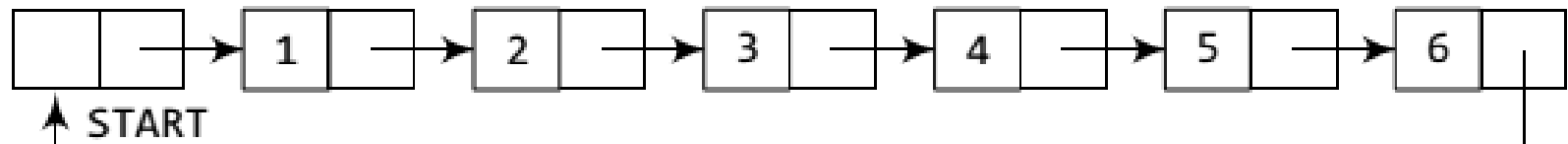
- Header Link List contains a header node at the beginning of the list.
- So, in a header linked list, START will not point to the first node of the list but START will contain the address of the header node.
- Header node, doesn't contain Data.
- Two Types** of Header Link List
 - 1. Grounded header linked list*** which stores NULL in the next field of the last node.
 - 2. Circular header linked list*** which stores the address of the header node in the next field of the last node

Header node



START

Header node



START

SPECIAL LINK LIST – 2. HEADER LINK LIST

- We can perform all operations, Insertion, Deletion and Traverse on Header Link List

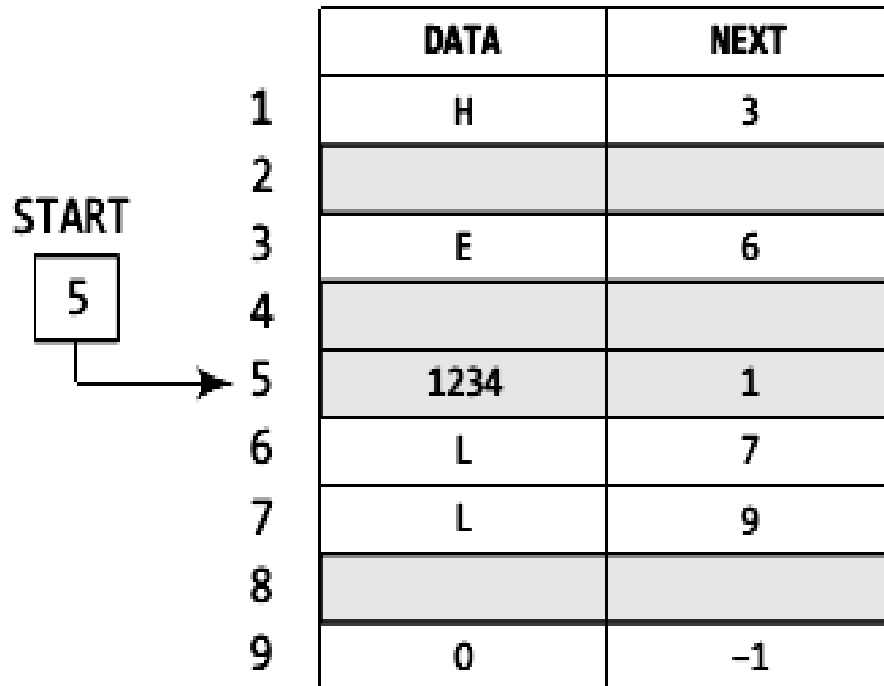


Figure 6.66 Memory representation of a header linked list

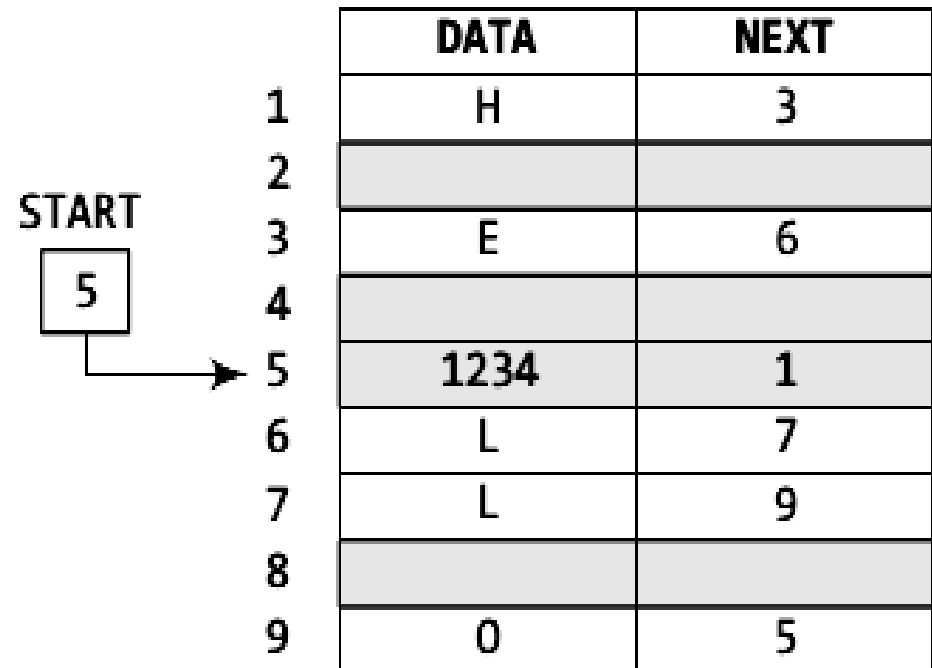


Figure 6.67 Memory representation of a circular header linked list

SPECIAL LINK LIST – 2. HEADER LINK LIST

- ALGORITHM: TRAVERSE_CIR_HEADER_LL(START)
 1. Set TEMP = START -> next
 2. While TEMP->next != START
 - PRINT TEMP->data
 - set TEMP = TEMP->next
 - end while

SPECIAL LINK LIST – 2. HEADER LINK LIST

- **ALGO:** INSERT_INBETWEEN_CIR_HEADER_LL(START, NUM)

1. *Declare PTR*

struct node * PTR

2. *Set* PTR = (struct node*) malloc (sizeof (struct node))

3. *Check Memory is AVAILABLE or Not*

if PTR == NULL then

WRITE “Memory is not available”

exit

end if

4. *Set* PTR -> data = VAL

PTR -> next = NULL

5. *If List is Empty*

IF START == NULL then

START->next = PTR

END IF

SPECIAL LINK LIST – 2. HEADER LINK LIST

5. *If List is NOT Empty*

TEMP = START -> next

while TEMP->data != NUM

TEMP = TEMP-> next

end while

6. *Add Node in the List*

PTR->next = TEMP->next

TEMP->next = PTR

7. *END*

The Magical Pond

You visit a shrine that has 3 temples in a magical pond.

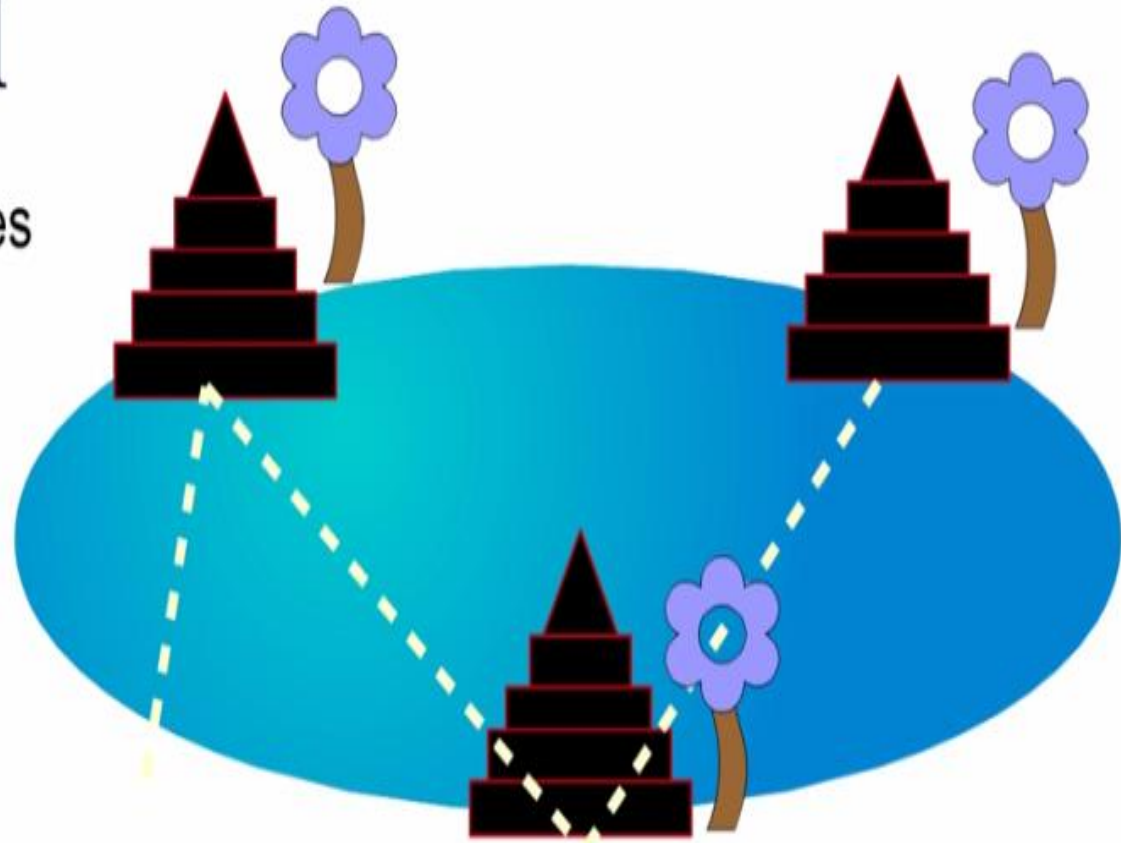
Before you visit a temple, you have to swim in the pond.

pond-temple1-pond-temple2-pond-temple3

Each time you swim in the pond, the number of flowers you are carrying magically triples.

You place an equal number of flowers at each temple you visit.

At the end, you have no flowers.



What is the least number of flowers you could have started with?

(You start with a positive number of flowers and you place a whole number of flowers at each temple.)



At the end, you have 0 flowers. So we have $27x - 13y = 0$, so $x = (13/27)y$.

To make a positive whole number, the least value of y is 27. This means $x = 13$.

